

Rendimiento y cadena 16.5–19 — todo lo hecho y encontrado el 2026-09-25

Qué es esto. El resumen de una jornada con tres frentes: acelerar el Python del proyecto, medir el paso 16.5 (variantes) de punta a punta con madres reales, y hacer lo mismo con el paso 19 (WFM). Todo medido en esta máquina (2 × EPYC 7413, 48 núcleos físicos / 96 lógicos, 125 GB), con la base re-medida el mismo día desde el commit anterior y cada cambio validado contra el código antiguo.

1. El resumen en una tabla

proceso	antes	después	cómo
<code>nulls.report</code> (757 estrategias × 4 peldaños)	569 s	3,6 s	numba, reparto por procesos, parquet leído una vez, caché de YAML
Monte Carlo (36 estrategias, 20.000 caminos)	477 s	27 s	una estrategia por proceso + kernel numba
crossmarket 499 × 9 mercados	38 min	4 min	reparto por (estrategia, mercado), LPT, numba
crossmarket 96 × 9	755 s	45 s (48 procesos)	ídem
<code>tasks.reports.filters</code> (10.000 estrategias)	38,7 s	3,2 s	candidatos repartidos
CSCV (962 variantes)	30,8 s	9,5 s	vecinos de la rejilla una vez; reglas en paralelo
crossTF	29,2 s	5,7 s	caché de YAML (54 de 60 s eran parsear)
<code>sqx.variants.make</code> (5.000)	11,2 s	2,1 s	fabricar en paralelo
<code>retest.ingest</code>	7,9 s · 2,9 GB	1,9 s · 1,9 GB	cada proceso escribe su P&L
<code>sppUltra</code>	7,0 s	1,9 s	prueba de inertes vectorizada
cosecha de variantes (<code>equity + collect</code>), por madre	581 s · 31,8 GB	100 s · 4,4 GB	lectura en paralelo, escritura por partes
vaciar el custodio entre madres	206 s · JVM 47 GB	3–4 s	borrar con SQX parado
<code>gate.harvest</code> (export 500+500)	114 s	81 s	un ciclo del conductor, no dos

Veinte procesos de Python medidos a la vez: 623 s → 67 s. Los diez más pequeños (WFM, WFC, parameterCloud, profitShape...) no cambiaron: su tiempo es importar pandas y scipy (~0,7 s).

2. Ronda 1 — numba y balanceo de carga en los nulos y crossmarket

Qué se hizo.

- Kernels numba que valoran y miden cada camino en una pasada, sin matrices intermedias: `nulls/kernel.py` , `strategies/crossmarket/simulate/kernel.py` .
- Barrido de barreras que se para en el primer toque.
- `core/fanout.py` : reparto por procesos con `fork` , **lo más caro primero (LPT)**, un hilo de BLAS por proceso (OpenBLAS arrancaba 64 en cada uno).
- crossmarket reparte por **(estrategia, mercado)** y calcula sólo lo que publica el `verdict.csv` .
- **Semilla estable** por (estrategia, peldaño, bloque): arregla que `nulls.seed` no fijaba nada (`hash()` salado por proceso; dos corridas daban 227 y 229 supervivientes).

Validación. `verdict.csv` de crossmarket idéntico byte a byte en 8×9 y 96×9, con 12 a 96 procesos. Kernels contra numpy con los mismos sorteos: $\leq 5,5 \cdot 10^{-11}$, drawdown y curvas idénticos. Los p de los nulos cambian por la semilla nueva, dentro del error Monte Carlo (0,23 % a más de 3 SE, se espera ~0,3 %).

Barrido del tamaño de bloque. El óptimo es **130–220 mil valoraciones de trade por bloque** (10–17 MB de temporales): el bloque se mide en trades, no en caminos. El 500 que había queda a un 0–13 % del óptimo en reposo; con la máquina cargada, el óptimo baja.

3. Dónde está ahora el cuello de botella (crossmarket)

- **No es el reparto:** 99 % de ocupación, la última tarea dura < 3 s, la parte en serie 1,5 s.
- **Cada tarea se vuelve más lenta cuantas más corren a la vez:** el mismo trabajo cuesta 1.155 s de CPU con 24 procesos, 1.940 con 48 y 5.219 con 96. De 48 a 96 se comparten núcleos (SMT); de 24 a 48 ya se comparte L3 y memoria.
- **Lo que peor lleva doblar de 48 a 96:** el kernel de los nulos (6,2× más lento) y el estrés de slippage (5,6×): los que leen precios a saltos.
- **Lo que más pesa hoy:** el test pareado (`paired.run`), el 42 % de cada tarea. Los nulos ya son el 12 %.
- **Usar 48 procesos, no 96.** Los comandos siguen con `os.cpu_count()` por defecto.

4. El paso 16.5 de punta a punta — tres madres reales

Tres madres de TestUSDJPY (`Strategy` 23.1.53 , 23.1.46 , 6.1.69), 5.000 variantes cada una, retesteadas en **3 tramos × 10 mercados** (USDJPY y su familia) = **150.000 backtests por madre**, en un proyecto propio del custodio (`USDJPY_variantes` , regla dura 10).

Por madre

fase	tiempo	memoria	disco
fabricar	3 s	—	67 MB locales
retest en SQX	63–71 min	JVM 84–87 GB (techo -Xmx80g)	7,5–14,7 GB en el custodio
cosecha (<code>equity</code> + <code>collect</code>)	~100 s	≤ 4,4 GB	—
vaciar el custodio	3–4 s	—	a 0
WFC	0,7 s	0,3 GB	—
CSCV	90–170 s	1,6 GB	—
se guarda			520–870 MB de parquet

Lo que la rompía — siete fallos, ninguno avisaba

- 1. La API no puede nombrar un databank con espacios (ni con %20 ni entre comillas): WFC Variants → WFC_Variants y los tres tramos, en assets/_build.yaml .
- 2. El proyecto no declaraba esos databanks y SQX los ignoraba; ni mkdir bastaba. Ahora los declara sqx.projects.wfc (cierra OPEN §40).
- 3. La tarea del tramo build traía una condición de OOS activa: todas las variantes fallaban y, con evaluateAll="false" , SQX no corría los mercados. Ahora se apagan todas.
- 4. Un proyecto sólo de retests no imprime Total tested ; el avance sale de In databank .
- 5. core.worker.holding() no veía los workers (./sqcli sin ruta): la guarda de la regla dura 4 estaba abierta en W1 y W2. Ahora mira también el directorio del proceso.
- 6. La clave de un resultado no es la carpeta del zip (/H1 frente a _LOM_H1).
- 7. Series.reset_index(names=...) , que pandas no admite, en un camino que nunca se había ejecutado.

Lo que se midió además

- El JVM va al límite en las tres madres, pero no revienta: el recolector libera en dientes de sierra. Sí frena el volcado final a disco (~15 min).
- Vaciar arrancando SQX era absurdo: al arrancar carga las 20.000 estrategias en memoria (47 GB) para después borrarlas. execute --clear borra con SQX parado, y la regla 1 juega a favor.
- Sin vaciar entre madres, 50 madres serían ~500 GB en el custodio.
- Validación de la cosecha nueva: en 200 ficheros de cada tramo, mismas curvas y mismas métricas que la lógica original.

5. El paso 19 (WFM) de punta a punta

Proyecto propio USDJPY_wfm en el custodio (la WFM del donante es Retest-Task2.xml), escrito con sqx.projects.wfm , sobre las mismas 3 madres. En modo mapa (min_squares: 0 , sólo en ese proyecto) para que las tres llegaran al análisis; la doctrina sigue filtrando con 12.

fase	3 madres	por madre
la matriz en SQX (30 celdas, 1.080 pasos)	13,2 min	~4–5 min, JVM 45 GB
exportar	20 s · 2,4 GB	~7 s
analizar	2 s	—

- Datos completos: 90 celdas, 1.080 pasos, 129.473 operaciones asignadas, 0 sin asignar.
- Dos fallos: export_wfm sólo sabía leer el maestro (ahora --role), y el pivote de los parámetros promediaba texto (ahora toma el valor y convierte lo numérico).
- SQX no da ninguna señal de avance en la WFM: Running time 0 ms y el databank quieto hasta que termina cada madre.
- Ruido inofensivo: la columna personalizada Param Count da un error cada vez que se lista un databank por la API.

6. La secuencia 16.5 → 19 por madre, y para 50

	por madre	50 madres
SQX (retest de variantes + WFM)	~70–75 min	~60–65 h
Python (cosecha, WFC, CSCV, export WFM, análisis)	~5 min	~4 h
se guarda	~0,6–0,9 GB	~30–45 GB

Lo que manda es SQX, y dentro de él el retest de las variantes. El Python ya no pesa.

7. Errores míos, dichos

- **Kernels numba con el modelo de errores por defecto:** 0/0 lanzaba excepción donde numpy da `nan`. Metido en la ronda 1, encontrado y arreglado en la 3 (`error_model="numpy"` en los tres).
- **Se perdió casi una hora** editando el script de medida mientras corría (bash lo lee sobre la marcha). No afectó a ninguna medida.
- **/tmp se llenó** (sólo tiene 3,9 GB) con copias de trabajo; limpiado, y las medidas pasaron al disco de datos.
- Mis primeras cifras de memoria de pools **sumaban RSS**, que con `fork` cuenta varias veces lo compartido. Desde la ronda 3, PSS.
- Mi primer pronóstico de que el JVM reventaría en la madre 1 era erróneo: tomé basura temporal por resultados guardados.

8. Decisiones tuyas registradas hoy

- Nulos: drawdown del mono en **orden de salida**; **swap según la duración sorteada** (con el multiplicador del día de la semana); **entradas restringidas al horario** de la estrategia; test **por estrategia**. Anotadas en `nulls/README.md`, **sin implementar**.
- Variantes y WFM en **proyecto propio**, nunca en el `Retester` de serie.
- Estudio con la **doctrina completa** (3 tramos × 10 mercados).
- **oos2 de USDJPY gastado** con madres de prueba: no leer esos resultados como un WFC/WFM real.
- WFM del test en **modo mapa**.

9. Lo que queda abierto

qué	por qué importa
El JVM del custodio toca su techo en cada madre de 5.000 variantes	una madre con más operaciones podría no caber: subir <code>-Xmx</code> o partir en tandas
<code>pipeline.run</code> no llama aún a <code>execute --clear</code> entre madres	hoy lo hace mi script de prueba
CSCV es lo más caro de Python (90–170 s por madre)	repartir varias madres a la vez
<code>equity_markets.parquet</code> son 450–710 MB por madre	el 80 % de lo que se guarda
El número de procesos por defecto es 96	con 48 va más rápido
Las métricas de la cosecha podrían salir del <code>.sqx</code> sin arrancar SQX	pierde Param Count, DoF y TRL: decisión tuya
Las cuatro decisiones de los nulos	escritas, sin programar
El pulso de los runs largos en la ventana	apuntado en el catálogo de la UI, §8.2

10. Dónde está cada cosa

- **Commits** (rama `perf/paralelizar-analisis`, sin push): `c9011a2` numba y balanceo · `baa5be3` el cuello de botella · `3923010` el resto de Python · `bfa8464` la cadena 16.5 · `694b6e1` sus medidas · `cf141ce` la WFM.
- **Medidas y datos**: `AlgoData/reports/perf-optim-2026-09-25/`, `AlgoData/profiling/bench-2026-09-25/`, `AlgoData/profiling/variantes-2026-09-25/real/` (las 3 madres, 1,9 GB), `AlgoData/raw/USDJPY_wfm/WFM/2026-09-25/`.
- **Instantáneas del custodio** antes de cada tanda: `AlgoData/snapshots/2026-09-25-w2-antes-*`.
- **Proyectos nuevos en el custodio**: `USDJPY_variantes` y `USDJPY_wfm`.
- **Knowhow**: `knowhow/sqx-drive/variant-chain-custom-project.md`, `wfm-end-to-end.md`, `knowhow/perf/` (`numba-division`, `yaml-parsing-cost`, `python-parallelism`).
- **Manual**: `12-rendimiento.md`, `19-wfc.md`, `26-nulos.md`, `39-crossmarket-lote.md`, `07-montecarlo.md`, `11-retest-mc.md`, `25-cscv.md`.